EAD
EDUCAÇÃO A DISTÂNCIA

UNIP
UNIVERSIDADE PAULISTA

Roteiros

Programação Orientada A Objetos

UNIP UNIVERSIDADE PAULISTA	**Disciplina: PROGRAMAÇÃO ORIENTADA A OBJETOS** **Título da aula: Introdução a Programação Orientada a Objetos**	**ROTEIRO 1**

1. Objetivos da aula

- Compreender o funcionamento básico de uma IDE para desenvolvimento em Java.
- Conhecer a estrutura de um projeto Java e seus principais componentes.
- Desenvolver o primeiro programa em Java utilizando entrada de dados pelo usuário.
- Familiarizar os alunos com conceitos iniciais de classes, métodos e execução de programas.

2. Recursos necessários

- Computadores com acesso à internet.
- IDE Java (NetBeans, IntelliJ IDEA ou Eclipse).
- Java Development Kit (JDK) instalado.
- Material de apoio da disciplina.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Apresentação do tema da aula e sua importância no aprendizado de programação.
- Explicação breve sobre a linguagem Java e sua utilização no mercado de tecnologia.
- Pergunta inicial para os alunos:

“Como um programa consegue interagir com o usuário e mostrar mensagens personalizadas?”

- Explicação do objetivo da aula: desenvolver o primeiro programa em Java com entrada de dados.

3.2. Revisão conceitual (20 minutos)

Breve revisão sobre conceitos fundamentais da programação:

- O que é uma linguagem de programação.
- O que é um programa de computador.
- Diferença entre compilação e execução.
- Estrutura básica de um programa Java:
    - Classe (class)
    - Método principal (main)

    - Comandos de saída (System.out.println)

Introdução rápida aos conceitos de Programação Orientada a Objetos:

- Classe
- Objeto
- Método

Explicação da importância da **IDE** no desenvolvimento de software.

3.3. Demonstração prática (30 minutos)

O professor realiza uma demonstração passo a passo.

Apresentação da interface da IDE

Explicar os principais elementos:

- Menu principal
- Área de edição de código
- Explorador de projetos
- Console de saída
- Botão de execução do programa

Criação de um novo projeto

Passos demonstrados:

1. Abrir a IDE
2. Criar um novo projeto Java
3. Criar uma nova classe chamada: OlaUsuario

Estrutura básica do programa Java

Explicação da estrutura do código:

```
public class OlaUsuario {
   public static void main(String[] args) {
   }
}
```

Inserção de entrada de dados

Utilizar a classe **Scanner** para ler dados do usuário.

Código completo demonstrado

```
import java.util.Scanner;
public class OlaUsuario {
   public static void main(String[] args) {
```

```
        Scanner entrada = new Scanner(System.in);
        System.out.println("Digite seu nome:");
        String nome = entrada.nextLine();
        System.out.println("Olá " + nome + ", seja bem-vindo ao Java!");
    }
}
```

Explicação do código

- import java.util.Scanner; → biblioteca para entrada de dados
- Scanner entrada → objeto que recebe dados do teclado
- nextLine() → lê texto digitado
- System.out.println() → exibe mensagens na tela

Executar o programa na IDE.

4. Atividade prática (40 minutos)

Desafio

Criar um programa Java que:

1. Solicite ao usuário:
    - Nome
2. Exiba uma mensagem personalizada:

Olá [nome digitado], bem-vindo ao mundo da programação em Java!

Passos que o aluno deve seguir

1. Criar um novo projeto Java.
2. Criar uma classe chamada:OlaUsuario

PrimeiroPrograma

3. Implementar o código de entrada e saída.
4. Executar o programa na IDE.
5. Testar com diferentes nomes.

Ampliação da atividade

Adicionar mais perguntas ao usuário, por exemplo:

- Idade
- Curso

Exemplo de saída:
Olá Maria!
Você tem 20 anos.
Bem-vinda ao curso de programação em Java.

5. Encerramento e orientações finais (20 minutos)

Discussão com os alunos:

- Dificuldades encontradas na criação do programa.
- Funcionamento da IDE e da execução do código.
- Importância da entrada de dados em programas interativos.

Reforçar conceitos aprendidos:

- Classe
- Método main
- Entrada de dados
- Saída de dados

Explicar a importância desse primeiro programa como base para estudos futuros em Programação Orientada a Objetos.

6. Orientações para o Relatório Final

Cada aluno deverá entregar um relatório contendo **1 a 3 páginas** com os seguintes itens:

Introdução teórica

Explicação sobre:

- O que é Java
- O que é uma IDE
- Estrutura básica de um programa Java

Descrição da atividade prática

Explicar:

- Como o programa foi criado
- Quais passos foram seguidos na IDE

Código comentado

O código deve conter comentários explicando cada parte.

Exemplo:

```
// Importa a biblioteca Scanner para entrada de dados
import java.util.Scanner;
public class OlaUsuario {
   public static void main(String[] args) {
      // Cria objeto para ler dados do teclado
      Scanner entrada = new Scanner(System.in);
```

```
        // Solicita o nome do usuário
        System.out.println("Digite seu nome:");

        // Armazena o nome digitado
        String nome = entrada.nextLine();

        // Exibe mensagem personalizada
        System.out.println("Olá " + nome);
    }

}
```

7. Critérios de avaliação

Critério	Peso	Descrição
Qualidade do resumo teórico	2,0	Clareza na explicação dos conceitos de Java e IDE
Funcionamento do programa	3,0	Programa executa corretamente com entrada e saída
Organização e clareza do código	3,0	Código estruturado e comentado
Criatividade e melhorias	2,0	Inclusão de novos campos ou melhorias

Nota final: soma dos critérios avaliados.

8. Conclusão

Ao final da aula, os alunos terão desenvolvido seu primeiro programa em Java, compreendendo a estrutura básica da linguagem, o funcionamento de uma IDE e a utilização de entrada e saída de dados. Essa atividade representa o primeiro passo para o desenvolvimento de aplicações mais complexas utilizando os princípios da Programação Orientada a Objetos.

UNIP UNIVERSIDADE PAULISTA	**Disciplina: PROGRAMAÇÃO ORIENTADA A OBJETOS** **Título da aula: Controle de Fluxo em Java: Estruturas Condicionais e Estruturas de Repetição**	**ROTEIRO 2**

1. Objetivos da aula

- Compreender o conceito de **controle de fluxo** em programas.
- Utilizar corretamente **estruturas condicionais** (if, else, else if, switch).
- Aplicar **estruturas de repetição** (for, while, do-while).
- Desenvolver algoritmos que tomem decisões e repitam ações.
- Resolver problemas práticos utilizando lógica condicional e laços.

2. Recursos necessários

- Computadores com acesso à internet.
- IDE Java (NetBeans, IntelliJ IDEA ou Eclipse).
- Java Development Kit (JDK) instalado.
- Material de apoio (slides, exemplos e roteiro).
- Projetor ou quadro para explicação.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Apresentação do tema:
  "Como fazer o programa tomar decisões e repetir tarefas automaticamente?"
- Contextualização:
  - Exemplo: sistema que verifica idade, calcula notas, repete operações.
- Pergunta norteadora:

"Como um programa decide o que fazer em diferentes situações?"

- Apresentação dos objetivos da aula.

3.2. Revisão conceitual (20 minutos)

Estruturas Condicionais

if / else

```
if (idade >= 18) {
    System.out.println("Maior de idade");
} else {
    System.out.println("Menor de idade");
}
```

else if

```
if (nota >= 7) {
    System.out.println("Aprovado");
} else if (nota >= 5) {
    System.out.println("Recuperação");
} else {
    System.out.println("Reprovado");
}
```

switch

```
switch (opcao) {
    case 1:
        System.out.println("Opção 1");
        break;
    case 2:
        System.out.println("Opção 2");
        break;
    default:
        System.out.println("Opção inválida");
}
```

Estruturas de Repetição

for (repetição com controle)

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

while (repetição com condição)

```
while (numero <= 5) {
    System.out.println(numero);
    numero++;
}
```

do-while (executa ao menos uma vez)

```
do {
    System.out.println(numero);
    numero++;
} while (numero <= 5);
```

Resumo conceitual

- Condicional → tomada de decisão
- Repetição → execução contínua
- Controle de fluxo → lógica do programa

3.3. Demonstração prática (30 minutos)

Passo a passo

1. Criar projeto Java.
2. Criar classe ControleFluxo.
3. Implementar exemplos de decisão e repetição.

Exemplo demonstrado

```
import java.util.Scanner;
public class ControleFluxo {
    public static void main(String[] args) {

        Scanner entrada = new Scanner(System.in);

        System.out.println("Digite sua idade:");
        int idade = entrada.nextInt();

        // Estrutura condicional
        if (idade >= 18) {
```

```
            System.out.println("Você é maior de idade.");
        } else {
            System.out.println("Você é menor de idade.");
        }

        // Estrutura de repetição
        System.out.println("\nContagem de 1 a 5:");
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
        }

        // Exemplo com while
        int contador = 1;
        while (contador <= 3) {
            System.out.println("Executando while: " + contador);
            contador++;
        }
    }
}
```

Explicação durante a prática

- Como o programa toma decisões
- Diferença entre os laços
- Quando usar for, while e do-while

4. Atividade prática (40 minutos)

Desafio principal

Criar um programa que:

1. Solicite ao usuário:
    - Nome
    - Idade
    - Nota
2. Realize:

- Verificação de maioridade (if/else)

- Classificação da nota:
    - ≥ 7 → Aprovado
    - ≥ 5 → Recuperação
    - < 5 → Reprovado
3. Exiba uma contagem de 1 até a idade do usuário.

Exemplo de saída

Nome: João

Maior de idade: Sim

Situação: Aprovado

Contagem até sua idade:

1 2 3 ... 25

Ampliação da atividade

Nível 1 – Condições múltiplas

- Adicionar validação:
    - Nota não pode ser negativa
    - Idade deve ser maior que zero

Nível 2 – Menu interativo (switch)

Criar menu:

1 - Verificar idade

2 - Calcular média

3 - Sair

Nível 3 – Repetição com controle

- Permitir que o usuário execute o programa várias vezes (while).

Nível 4 – Contadores e acumuladores

- Calcular média de várias notas inseridas pelo usuário.

Nível 5 – Desafio extra

- Criar sistema que:

    - Leia 5 alunos
    - Exiba quantos foram aprovados

5. Encerramento e orientações finais (20 minutos)

Discussão:

- Diferença entre decisão e repetição
- Situações reais de uso

Pergunta reflexiva:

“Como estruturas de controle tornam os programas inteligentes?”

Reforço:

- if/else
- switch
- for, while, do-while

6. Orientações para o Relatório Final

O aluno deverá entregar um relatório (1 a 3 páginas):

Introdução

- Conceito de controle de fluxo
- Estruturas condicionais e de repetição

Desenvolvimento

- Explicação do programa
- Estruturas utilizadas

Código comentado

```
// Verifica se é maior de idade
if (idade >= 18) {
   System.out.println("Maior de idade");
}
```

Resultados

- Saída do programa

Conclusão

- Aprendizados obtidos

7. Critérios de avaliação

Critério	Peso	Descrição
Fundamentação teórica	2,0	Explicação das estruturas
Implementação prática	3,0	Uso correto de if e loops
Organização do código	2,0	Código estruturado
Execução e testes	2,0	Funcionamento correto
Criatividade	1,0	Melhorias e expansão

Nota total: soma dos critérios avaliados.

8. Conclusão

Ao final da aula, os alunos estarão aptos a utilizar **estruturas condicionais e de repetição em Java**, compreendendo como controlar o fluxo de execução de um programa. Esses conceitos são fundamentais para o desenvolvimento de soluções mais complexas, sendo base para algoritmos, sistemas interativos e aplicações reais.

UNIP UNIVERSIDADE PAULISTA	**Disciplina: PROGRAMAÇÃO ORIENTADA A OBJETOS** **Título da aula: Programação Orientada a Objetos em Java: Encapsulamento, Construtores, Sobrecarga, Herança e Sobrescrita**	**ROTEIRO 3**

1. Objetivos da aula

- Compreender o conceito de **encapsulamento** e sua importância na proteção dos dados.
- Aplicar **métodos construtores** para inicialização de objetos.
- Implementar **sobrecarga de métodos**.
- Entender o conceito de **herança** entre classes.
- Aplicar **sobrescrita de métodos (override)**.
- Desenvolver soluções orientadas a objetos utilizando boas práticas.

2. Recursos necessários

- Computadores com acesso à internet.
- IDE Java (NetBeans, IntelliJ IDEA ou Eclipse).
- Java Development Kit (JDK) instalado.
- Material de apoio (slides, exemplos e roteiro).
- Projetor ou quadro para explicação.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Apresentação do tema:

  “Como organizar melhor o código e reutilizar funcionalidades em Java?”

- Contextualização:
  - Exemplo: sistema de cadastro com diferentes tipos de usuários.
- Pergunta norteadora:

“Como podemos proteger dados e reaproveitar código em um sistema?”

- Apresentação dos objetivos da aula.

3.2. Revisão conceitual (20 minutos)

Encapsulamento

- Uso de modificadores de acesso (private).
- Métodos getters e setters.

```
private String nome;

public String getNome() {
    return nome;
}

public void setNome(String nome) {
    this.nome = nome;
}
```

Método Construtor

- Inicializa objetos.
- Não possui tipo de retorno.

```
public Pessoa(String nome) {
    this.nome = nome;
}
```

Sobrecarga de métodos (Overload)

- Métodos com mesmo nome, mas parâmetros diferentes.

```
public void exibir() {
    System.out.println("Sem parâmetros");
}

public void exibir(String nome) {
    System.out.println("Nome: " + nome);
}
```

Herança

- Reutilização de código com extends.

```
class Aluno extends Pessoa {
}
```

Sobrescrita (Override)

- Alteração de comportamento de método herdado.

```
@Override
public void exibir() {
    System.out.println("Exibição do aluno");
}
```

Resumo conceitual

- Encapsulamento → proteção de dados
- Construtor → inicialização
- Sobrecarga → flexibilidade
- Herança → reutilização
- Sobrescrita → especialização

3.3. Demonstração prática (30 minutos)

Passo a passo

1. Criar projeto Java.
2. Criar classe Pessoa.
3. Criar classe Aluno herdando de Pessoa.

Classe base (Pessoa)

```
class Pessoa {

    private String nome;
    private int idade;

    // Construtor
    public Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    // Getters e Setters
    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }
```

```
    // Método
    public void exibirDados() {
        System.out.println("Nome: " + nome);
        System.out.println("Idade: " + idade);
    }
}
```

Classe derivada (Aluno)

```
class Aluno extends Pessoa {

    private String curso;

    public Aluno(String nome, int idade, String curso) {
        super(nome, idade);
        this.curso = curso;
    }

    // Sobrescrita
    @Override
    public void exibirDados() {
        super.exibirDados();
        System.out.println("Curso: " + curso);
    }
}
```

Classe principal

```
public class Main {

    public static void main(String[] args) {

        Aluno aluno = new Aluno("Maria", 20, "ADS");

        aluno.exibirDados();
    }
}
```

Explicação durante a prática

- Uso de private e encapsulamento
- Uso de super()
- Diferença entre overload e override
- Hierarquia de classes

4. Atividade prática (40 minutos)

Desafio principal

Criar um sistema com:

Classe base: Funcionario

- Atributos privados:
  - nome
  - salario
- Métodos:
  - getters e setters
  - exibirDados()

Classe derivada: Gerente

- Atributo:
  - bonus
- Implementar:
  - Construtor com super
  - Sobrescrita do método exibirDados()

Requisitos

- Criar pelo menos 2 objetos.
- Demonstrar herança e sobrescrita.
- Aplicar encapsulamento corretamente.

Ampliação da atividade

Nível 1 – Encapsulamento completo

- Tornar todos os atributos privados.
- Criar validações nos setters.

Nível 2 – Sobrecarga

- Criar múltiplos construtores.
- Criar métodos com diferentes parâmetros.

Nível 3 – Hierarquia

- Criar nova classe Estagiario.

Nível 4 – Polimorfismo simples

- Utilizar referência da classe base:

Funcionario f = new Gerente(...);

Nível 5 – Desafio extra

- Criar cálculo de salário com bônus.
- Criar menu interativo.

5. Encerramento e orientações finais (20 minutos)

Discussão:

- Diferença entre herança e encapsulamento
- Importância da reutilização de código

Pergunta reflexiva:

“Como a herança reduz a repetição de código em sistemas reais?”

Reforço:

- Encapsulamento
- Construtores
- Sobrecarga e sobrescrita

6. Orientações para o Relatório Final

O aluno deverá entregar um relatório (1 a 3 páginas):

Introdução

- Conceitos de POO
- Encapsulamento e herança

Desenvolvimento

- Explicação das classes criadas
- Hierarquia implementada

Código comentado

```
// Método sobrescrito
@Override
```

```
public void exibirDados() {
    super.exibirDados();
}
```

Resultados

- Saída do programa

Conclusão

- Aprendizados

7. Critérios de avaliação

Critério	Peso	Descrição
Fundamentação teórica	2,0	Conceitos de POO
Implementação prática	3,0	Uso correto de herança e encapsulamento
Uso de métodos	2,0	Sobrecarga e sobrescrita
Organização do código	2,0	Estrutura e clareza
Criatividade	1,0	Melhorias

Nota total: soma dos critérios avaliados.

8. Conclusão

Ao final da aula, os alunos terão compreendido e aplicado os principais pilares da **Programação Orientada a Objetos em Java**, incluindo encapsulamento, construtores, herança, sobrecarga e sobrescrita. Esses conceitos são fundamentais para o desenvolvimento de sistemas mais organizados, reutilizáveis e escaláveis.

UNIP UNIVERSIDADE PAULISTA	**Disciplina: PROGRAMAÇÃO ORIENTADA A OBJETOS** **Título da aula: Estruturas Dinâmicas em Java: ArrayList e Vector**	**ROTEIRO 4**

1. Objetivos da aula

- Compreender o conceito de **coleções dinâmicas** em Java.
- Diferenciar **ArrayList** e **Vector**.
- Utilizar métodos principais das listas:
  - add(), remove(), get(), set(), size()
- Entender o conceito de **tipagem genérica (Generics)**.
- Aplicar listas dinâmicas em programas reais.

2. Recursos necessários

- Computadores com acesso à internet.
- IDE Java (NetBeans, IntelliJ IDEA ou Eclipse).
- Java Development Kit (JDK) instalado.
- Material de apoio (slides, exemplos e roteiro).
- Projetor ou quadro para explicação.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Apresentação do tema:
  "Como armazenar uma quantidade variável de dados em Java?"
- Contextualização:
  - Exemplo: lista de alunos, produtos ou notas.
- Pergunta norteadora:

"E se não soubermos quantos dados teremos no programa?"

- Apresentação dos objetivos da aula.

3.2. Revisão conceitual (20 minutos)

Arrays vs ArrayList

- Arrays → tamanho fixo
- ArrayList → tamanho dinâmico

ArrayList

- Parte do pacote java.util
- Não é sincronizado (mais rápido)
- Uso mais comum

```
import java.util.ArrayList;
```

Exemplo:

```
ArrayList<String> nomes = new ArrayList<>();
nomes.add("Maria");
nomes.add("João");
```

Vector

- Também do pacote java.util
- Sincronizado (thread-safe)
- Menos utilizado atualmente

```
import java.util.Vector;
```

Principais métodos

Método	Função
add()	Adiciona elemento
remove()	Remove elemento
get()	Acessa elemento
set()	Altera elemento
size()	Retorna tamanho

Generics (tipagem)

```
ArrayList<Integer> numeros = new ArrayList<>();
```

Resumo conceitual

- ArrayList → mais rápido, uso geral
- Vector → seguro para threads
- Ambos → listas dinâmicas

3.3. Demonstração prática (30 minutos)

Passo a passo

1. Criar projeto Java.
2. Criar classe ListaExemplo.
3. Importar ArrayList e Vector.

Exemplo com ArrayList

```
import java.util.ArrayList;
public class ListaExemplo {
    public static void main(String[] args) {

        ArrayList<String> alunos = new ArrayList<>();

        alunos.add("Maria");
        alunos.add("João");
        alunos.add("Ana");

        System.out.println("Lista de alunos:");

        for (int i = 0; i < alunos.size(); i++) {
            System.out.println(alunos.get(i));
        }

        alunos.remove("João");

        System.out.println("\nApós remoção:");
        for (String aluno : alunos) {
            System.out.println(aluno);
        }
    }
}
```

Exemplo com Vector

```
import java.util.Vector;
```

```
Vector<Integer> numeros = new Vector<>();

numeros.add(10);
numeros.add(20);
numeros.add(30);

System.out.println("Total de elementos: " + numeros.size());
```

Explicação durante a prática

- Diferença entre ArrayList e Vector
- Uso de laços (for / foreach)
- Manipulação de elementos

4. Atividade prática (40 minutos)

Desafio principal

Criar um programa que:

1. Armazene nomes de alunos usando **ArrayList**.
2. Permita:
    - Inserir nomes
    - Exibir lista
    - Remover um nome

Requisitos

- Utilizar pelo menos 3 métodos (add, remove, get).
- Exibir lista antes e depois da remoção.

Exemplo de saída

```
Alunos:
Maria
João
Ana

Removendo João...
```

Lista atualizada:

Maria

Ana

Ampliação da atividade

Nível 1 – Entrada dinâmica

- Utilizar Scanner para inserir dados.

Nível 2 – Lista de números

- Criar lista de números (ArrayList<Integer>).
- Calcular soma e média.

Nível 3 – Uso de Vector

- Repetir atividade usando Vector.
- Comparar comportamento.

Nível 4 – Menu interativo

1 - Adicionar

2 - Remover

3 - Listar

4 - Sair

Nível 5 – Desafio extra

- Criar lista de objetos (ex: classe Aluno).
- Armazenar objetos no ArrayList.

```
ArrayList<Aluno> lista = new ArrayList<>();
```

5. Encerramento e orientações finais (20 minutos)

Discussão:

- Quando usar ArrayList ou Vector
- Diferença entre listas e arrays

Pergunta reflexiva:

“Por que listas dinâmicas são mais utilizadas em sistemas reais?”

Reforço:

- Estruturas dinâmicas
- Manipulação de dados
- Generics

6. Orientações para o Relatório Final

O aluno deverá entregar um relatório (1 a 3 páginas):

Introdução

- Conceito de listas dinâmicas
- Diferença entre ArrayList e Vector

Desenvolvimento

- Explicação do programa
- Métodos utilizados

Código comentado

```
// Adiciona elemento na lista
alunos.add("Maria");
```

Resultados

- Saídas do programa

Conclusão

- Aprendizados adquiridos

7. Critérios de avaliação

Critério	Peso	Descrição
Fundamentação teórica	2,0	Conceitos de listas
Implementação prática	3,0	Uso correto do ArrayList/Vector
Manipulação de dados	2,0	Uso de métodos
Organização do código	2,0	Clareza e estrutura
Criatividade	1,0	Melhorias

Nota total: soma dos critérios avaliados.

8. Conclusão

Ao final da aula, os alunos estarão aptos a utilizar **ArrayList e Vector em Java**, compreendendo suas diferenças e aplicações. Esses conceitos são fundamentais para o desenvolvimento de sistemas que manipulam grandes volumes de dados de forma dinâmica e eficiente.

UNIP UNIVERSIDADE PAULISTA	**Disciplina: PROGRAMAÇÃO ORIENTADA A OBJETOS** **Título da aula: Polimorfismo em Java: Flexibilidade com Variáveis, Métodos e Classes**	**ROTEIRO 5**

1. Objetivos da aula

- Compreender o conceito de **polimorfismo** na Programação Orientada a Objetos.
- Diferenciar:
  - Polimorfismo de variáveis
  - Polimorfismo de métodos (sobrecarga e sobrescrita)
  - Polimorfismo de classes (herança)
- Aplicar **polimorfismo em tempo de compilação (overload)** e **tempo de execução (override)**.
- Desenvolver programas mais flexíveis e reutilizáveis.

2. Recursos necessários

- Computadores com acesso à internet.
- IDE Java (NetBeans, IntelliJ IDEA ou Eclipse).
- Java Development Kit (JDK) instalado.
- Material de apoio (slides, exemplos e roteiro).
- Projetor ou quadro para explicação.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Apresentação do tema:
  "Como um mesmo método pode ter diferentes comportamentos?"
- Contextualização:
  - Exemplo: diferentes tipos de pagamento, veículos ou usuários.
- Pergunta norteadora:

"Como reutilizar código mantendo comportamentos diferentes?"

- Apresentação dos objetivos da aula.

3.2. Revisão conceitual (20 minutos)

O que é Polimorfismo

- Capacidade de um objeto assumir diferentes formas.

Polimorfismo de métodos (Sobrecarga – Overload)

- Mesmo nome, parâmetros diferentes.

```
public void somar(int a, int b) {
   System.out.println(a + b);
}

public void somar(double a, double b) {
   System.out.println(a + b);
}
```

Polimorfismo de métodos (Sobrescrita – Override)

- Método redefinido na classe filha.

```
@Override
public void exibir() {
   System.out.println("Classe filha");
}
```

Polimorfismo de variáveis

- Referência da classe pai apontando para objeto da classe filha.

```
Pessoa p = new Aluno();
```

Polimorfismo de classes

- Uso de herança (extends).

Resumo conceitual

- Overload → tempo de compilação
- Override → tempo de execução
- Referência → flexibilidade
- Herança → base do polimorfismo

3.3. Demonstração prática (30 minutos)

Passo a passo

1. Criar classe base Animal.
2. Criar classes derivadas Cachorro e Gato.
3. Implementar sobrescrita de métodos.

Classe base

```
class Animal {

    public void emitirSom() {
        System.out.println("Som genérico");
    }
}
```

Classes derivadas

```
class Cachorro extends Animal {

    @Override
    public void emitirSom() {
        System.out.println("Latido");
    }
}

class Gato extends Animal {

    @Override
    public void emitirSom() {
        System.out.println("Miado");
    }
}
```

Classe principal (polimorfismo)

```
public class Main {
    public static void main(String[] args) {

        Animal a1 = new Cachorro();
```

```
        Animal a2 = new Gato();

        a1.emitirSom();
        a2.emitirSom();
    }
}
```

Exemplo de sobrecarga

```
public class Calculadora {

    public int soma(int a, int b) {
        return a + b;
    }

    public double soma(double a, double b) {
        return a + b;
    }
}
```

Explicação durante a prática

- Diferença entre overload e override
- Uso de referência genérica
- Comportamento dinâmico

4. Atividade prática (40 minutos)

Desafio principal

Criar um sistema com:

Classe base: Funcionario

- Método:
    - calcularSalario()

Classes derivadas

- Gerente
- Desenvolvedor

Cada uma deve sobrescrever o método com lógica diferente.

Requisitos

- Criar objetos usando referência da classe base:

Funcionario f = new Gerente();

- Demonstrar comportamento diferente.

Exemplo esperado

Gerente: Salário com bônus

Desenvolvedor: Salário padrão

Ampliação da atividade

Nível 1 – Sobrescrita completa

- Criar pelo menos 3 classes derivadas.

Nível 2 – Sobrecarga

- Criar métodos com parâmetros diferentes.

Nível 3 – Polimorfismo em lista

ArrayList<Funcionario> lista = new ArrayList<>();

- Armazenar diferentes tipos de funcionários.

Nível 4 – Método genérico

- Criar método que receba Funcionario e execute ações.

Nível 5 – Desafio extra

- Criar menu interativo.
- Aplicar decisões com if ou switch.

5. Encerramento e orientações finais (20 minutos)

Discussão:

- Vantagens do polimorfismo
- Reutilização de código

Pergunta reflexiva:

“Por que o polimorfismo torna o código mais flexível?”

Reforço:

- Overload vs Override
- Herança
- Referência genérica

6. Orientações para o Relatório Final

O aluno deverá entregar um relatório (1 a 3 páginas):

Introdução

- Conceito de polimorfismo
- Tipos de polimorfismo

Desenvolvimento

- Explicação do sistema desenvolvido
- Classes criadas

Código comentado

```
// Polimorfismo
Funcionario f = new Gerente();
```

Resultados

- Saída do programa

Conclusão

- Aprendizados

7. Critérios de avaliação

Critério	Peso	Descrição
Fundamentação teórica	2,0	Conceitos de polimorfismo
Implementação prática	3,0	Uso de override e overload
Uso de herança	2,0	Estrutura correta
Organização do código	2,0	Clareza
Criatividade	1,0	Melhorias

Nota total: soma dos critérios avaliados.

8. Conclusão

Ao final da aula, os alunos estarão aptos a compreender e aplicar o **polimorfismo em Java**, utilizando diferentes formas de implementação (variáveis, métodos e classes). Esse conceito é essencial para o desenvolvimento de sistemas flexíveis, reutilizáveis e escaláveis, sendo um dos pilares mais importantes da Programação Orientada a Objetos.

UNIP UNIVERSIDADE PAULISTA	**Disciplina: PROGRAMAÇÃO ORIENTADA A OBJETOS** **Título da aula: Modificadores de Comportamento em Java: final, static e abstract**	**ROTEIRO 6**

1. Objetivos da aula

- Compreender o papel dos **modificadores de comportamento** em Java.
- Diferenciar e aplicar:
  - final
  - static
  - abstract
- Entender como esses modificadores impactam:
  - Variáveis
  - Métodos
  - Classes
- Desenvolver código mais **seguro, organizado e reutilizável**.

2. Recursos necessários

- Computadores com acesso à internet.
- IDE Java (NetBeans, IntelliJ IDEA ou Eclipse).
- Java Development Kit (JDK) instalado.
- Material de apoio (slides, exemplos e roteiro).
- Projetor ou quadro para explicação.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Apresentação do tema:
  "Como controlar o comportamento e as regras de um programa em Java?"
- Contextualização:
  - Exemplo: constantes, métodos compartilhados, classes base.
- Pergunta norteadora:

"Como evitar alterações indevidas e organizar melhor o código?"

- Apresentação dos objetivos da aula.

3.2. Revisão conceitual (20 minutos)

Modificador final

- Impede alteração.

Aplicações:

- Variável → valor constante
- Método → não pode ser sobrescrito
- Classe → não pode ser herdada

```
final double PI = 3.14;
```

Modificador static

- Pertence à classe (não ao objeto).

Aplicações:

- Variáveis compartilhadas
- Métodos utilitários

```
static int contador = 0;
```

Modificador abstract

- Define estrutura base.
- Não pode ser instanciado.

```
abstract class Animal {
   abstract void emitirSom();
}
```

Resumo conceitual

- final → proteção
- static → compartilhamento
- abstract → generalização

3.3. Demonstração prática (30 minutos)

Passo a passo

1. Criar projeto Java.
2. Criar classe abstrata.
3. Criar classes concretas.
4. Demonstrar uso de static e final.

Exemplo completo

Classe abstrata

```
abstract class Animal {

    // Método abstrato
    abstract void emitirSom();

    // Método concreto
    public void dormir() {
        System.out.println("Animal dormindo...");
    }
}
```

Classe concreta

```
class Cachorro extends Animal {

    @Override
    void emitirSom() {
        System.out.println("Latido");
    }
}
```

Uso de static e final

```
class Util {
    static int contador = 0;
    final double TAXA = 1.5;
    public static void incrementar() {
        contador++;
    }
}
```

Classe principal

```
public class Main {
    public static void main(String[] args) {

        Animal a = new Cachorro();
        a.emitirSom();
        a.dormir();

        Util.incrementar();
        System.out.println("Contador: " + Util.contador);
    }
}
```

Explicação durante a prática

- Diferença entre classe abstrata e concreta
- Uso de métodos estáticos
- Importância de constantes (final)
- Regras de herança

4. Atividade prática (40 minutos)

Desafio principal

Criar um sistema com:

Classe abstrata: Funcionario

- Método abstrato:
    - calcularSalario()

Classes derivadas

- Gerente
- Vendedor

Cada uma deve:

- Implementar o método abstrato
- Calcular salário de forma diferente

Uso obrigatório

- Variável static para contar funcionários
- Variável final para taxa fixa

Requisitos

- Criar pelo menos 2 objetos

- Demonstrar uso dos três modificadores

Ampliação da atividade

Nível 1 – Uso completo de final

- Criar constantes:
    - Salário mínimo
    - Taxa de bônus

Nível 2 – Métodos estáticos

- Criar métodos utilitários (ex: cálculo de imposto)

Nível 3 – Hierarquia abstrata

- Criar nova classe derivada (ex: Estagiario)

Nível 4 – Regras de negócio

- Impedir alteração de valores com final
- Garantir implementação com abstract

Nível 5 – Desafio extra

- Criar menu interativo
- Aplicar validações

5. Encerramento e orientações finais (20 minutos)

Discussão:

- Quando usar cada modificador
- Impacto na organização do código

Pergunta reflexiva:

“Como esses modificadores ajudam na qualidade do software?”

Reforço:

- final → segurança
- static → compartilhamento
- abstract → estrutura

6. Orientações para o Relatório Final

O aluno deverá entregar um relatório (1 a 3 páginas):

Introdução

- Conceito de modificadores
- Importância na POO

Desenvolvimento

- Explicação do sistema criado
- Uso de cada modificador

Código comentado

```
// Variável constante
final double TAXA = 1.5;
```

Resultados

- Saída do programa

Conclusão

- Aprendizados adquiridos

7. Critérios de avaliação

Critério	Peso	Descrição
Fundamentação teórica	2,0	Entendimento dos modificadores
Implementação prática	3,0	Uso correto de final, static, abstract
Estrutura do código	2,0	Organização
Funcionamento	2,0	Execução correta
Criatividade	1,0	Melhorias

Nota total: soma dos critérios avaliados.

8. Conclusão

Ao final da aula, os alunos estarão aptos a compreender e aplicar os **modificadores de comportamento em Java**, utilizando final, static e abstract para construir sistemas mais organizados, seguros e reutilizáveis. Esses conceitos são fundamentais para o desenvolvimento profissional e para a criação de arquiteturas de software bem estruturadas.

UNIP UNIVERSIDADE PAULISTA	**Disciplina: PROGRAMAÇÃO ORIENTADA A OBJETOS** **Título da aula: Tratamento de Exceções em Java: Garantindo Robustez e Segurança no Código**	**ROTEIRO 7**

1. Objetivos da aula

- Compreender o conceito de **exceções** em Java.
- Diferenciar:
  - Erros de compilação
  - Erros de execução (exceptions)
- Utilizar corretamente:
  - try, catch, finally
  - throw e throws
- Identificar exceções comuns:
  - ArithmeticException
  - NullPointerException
  - InputMismatchException
- Desenvolver programas mais **seguros e resilientes**.

2. Recursos necessários

- Computadores com acesso à internet.
- IDE Java (NetBeans, IntelliJ IDEA ou Eclipse).
- Java Development Kit (JDK) instalado.
- Material de apoio (slides, exemplos e roteiro).
- Projetor ou quadro para explicação.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Apresentação do tema:
  “O que acontece quando um programa encontra um erro inesperado?”
- Contextualização:
  - Exemplo: divisão por zero, entrada inválida, variável nula.
- Pergunta norteadora:

“Como evitar que um programa seja interrompido por erros?”

- Apresentação dos objetivos da aula.

3.2. Revisão conceitual (20 minutos)

O que são exceções

- Eventos que interrompem o fluxo normal do programa.

Estrutura básica

```
try {
    // código que pode gerar erro
} catch (Exception e) {
    // tratamento do erro
}
```

Uso do finally

```
finally {
    // executa sempre
}
```

Exceções comuns

- ArithmeticException → divisão por zero
- NullPointerException → objeto nulo
- InputMismatchException → tipo inválido

throw e throws

```
throw new Exception("Erro personalizado");
```

Resumo conceitual

- try → tenta executar
- catch → trata erro
- finally → sempre executa
- throw → lança erro

3.3. Demonstração prática (30 minutos)

Passo a passo

1. Criar projeto Java.
2. Criar classe Excecoes.
3. Simular erros e tratar exceções.

Exemplo prático

```
import java.util.Scanner;
import java.util.InputMismatchException;
public class Excecoes {
    public static void main(String[] args) {

        Scanner entrada = new Scanner(System.in);

        try {
            System.out.println("Digite um número:");
            int numero = entrada.nextInt();

            int resultado = 10 / numero;

            System.out.println("Resultado: " + resultado);

        } catch (ArithmeticException e) {
            System.out.println("Erro: divisão por zero!");

        } catch (InputMismatchException e) {
            System.out.println("Erro: entrada inválida!");

        } finally {
            System.out.println("Programa finalizado.");
        }
    }
}
```

Exemplo com throw

```
public static void validarIdade(int idade) throws Exception {
    if (idade < 0) {
        throw new Exception("Idade inválida!");
    }
}
```

Explicação durante a prática

- Fluxo de execução com exceções
- Ordem dos blocos catch
- Importância do tratamento

4. Atividade prática (40 minutos)

Desafio principal

Criar um programa que:

1. Solicite ao usuário:
    - Nome
    - Idade
    - Número para divisão
2. Implemente:

- Tratamento de entrada inválida
- Tratamento de divisão por zero
- Validação de idade (não negativa)

Requisitos

- Utilizar try, catch e finally
- Utilizar pelo menos 2 tipos de exceção
- Implementar um throw personalizado

Exemplo de saída

Erro: entrada inválida!

Erro: divisão por zero!

Erro: idade inválida!

Ampliação da atividade

Nível 1 – Múltiplos catch

- Tratar diferentes exceções separadamente.

Nível 2 – Exceção personalizada

Criar classe:

```
class IdadeInvalidaException extends Exception {
}
```

Nível 3 – Método com throws

- Criar método que lança exceção.

Nível 4 – Validação completa

- Validar todos os dados de entrada.

Nível 5 – Desafio extra

- Criar menu interativo
- Repetir execução até dados válidos

5. Encerramento e orientações finais (20 minutos)

Discussão:

- Importância do tratamento de erros
- Diferença entre erro e exceção

Pergunta reflexiva:

“Como exceções ajudam a evitar falhas em sistemas reais?”

Reforço:

- try/catch/finally
- tipos de exceção
- validação

6. Orientações para o Relatório Final

O aluno deverá entregar um relatório (1 a 3 páginas):

Introdução

- Conceito de exceções
- Importância do tratamento

Desenvolvimento

- Explicação do programa
- Tipos de exceções utilizadas

Código comentado

```
// Trata erro de divisão por zero
catch (ArithmeticException e) {
   System.out.println("Erro!");
}
```

Resultados

- Saída do programa

Conclusão

- Aprendizados adquiridos

7. Critérios de avaliação

Critério	Peso	Descrição
Fundamentação teórica	2,0	Conceitos de exceções
Implementação prática	3,0	Uso correto de try/catch
Tratamento de erros	2,0	Validação adequada
Organização do código	2,0	Clareza
Criatividade	1,0	Melhorias

Nota total: soma dos critérios avaliados.

8. Conclusão

Ao final da aula, os alunos estarão aptos a compreender e aplicar o **tratamento de exceções em Java**, desenvolvendo programas mais robustos, seguros e preparados para lidar com erros. Esse conhecimento é essencial para aplicações reais, onde falhas devem ser tratadas de forma controlada e eficiente.

UNIP UNIVERSIDADE PAULISTA	**Disciplina: PROGRAMAÇÃO ORIENTADA A OBJETOS** **Título da aula: Programação Concorrente em Java: Threads e Execução Paralela**	**ROTEIRO 8**

. **1. Objetivos da aula**

- Compreender o conceito de **threads (processos leves)** em Java.
- Diferenciar execução **sequencial vs concorrente**.
- Criar threads utilizando:
  - Classe Thread
  - Interface Runnable
- Controlar execução com:
  - start(), run(), sleep()
- Entender problemas básicos de concorrência.
- Desenvolver aplicações com execução paralela.

2. Recursos necessários

- Computadores com acesso à internet.
- IDE Java (NetBeans, IntelliJ IDEA ou Eclipse).
- Java Development Kit (JDK) instalado.
- Material de apoio (slides, exemplos e roteiro).
- Projetor ou quadro para explicação.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Apresentação do tema:

  "Como um programa pode executar várias tarefas ao mesmo tempo?"

- Contextualização:
  - Exemplo: download + música + processamento.
- Pergunta norteadora:

"Como dividir tarefas para ganhar desempenho?"

- Apresentação dos objetivos da aula.

3.2. Revisão conceitual (20 minutos)

O que são Threads

- Fluxos de execução dentro de um programa.

Execução sequencial vs concorrente

- Sequencial → uma tarefa por vez
- Concorrente → múltiplas tarefas

Criando Threads

1. Herdando Thread

```
class MinhaThread extends Thread {
    public void run() {
        System.out.println("Executando thread");
    }
}
```

2. Implementando Runnable

```
class MinhaTarefa implements Runnable {
    public void run() {
        System.out.println("Executando tarefa");
    }
}
```

Métodos importantes

- start() → inicia thread
- run() → lógica
- sleep() → pausa

Resumo conceitual

- Thread → execução paralela
- Runnable → mais flexível
- start() → inicia execução real

3.3. Demonstração prática (30 minutos)

Passo a passo

1. Criar projeto Java.
2. Criar classe de thread.
3. Executar múltiplas threads.

Exemplo com Thread

```
class Contador extends Thread {

    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + " - " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println("Erro na thread");
            }
        }
    }
}
```

Classe principal

```
public class Main {

    public static void main(String[] args) {

        Contador t1 = new Contador();
        Contador t2 = new Contador();

        t1.start();
        t2.start();
    }
}
```

Exemplo com Runnable

```
class Tarefa implements Runnable {

    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Runnable: " + i);
        }
    }
}
```

Explicação durante a prática

- Diferença entre run() e start()
- Execução simultânea
- Interleaving de saída

4. Atividade prática (40 minutos)

Desafio principal

Criar um programa que:

1. Execute **duas threads simultaneamente**:
    - Thread 1: conta de 1 a 10
    - Thread 2: conta de 10 a 1
2. Utilize sleep() para simular tempo de execução.

Requisitos

- Criar pelo menos 2 threads.
- Demonstrar execução concorrente.
- Utilizar start() corretamente.

Exemplo de saída

```
Thread-0: 1
Thread-1: 10
Thread-0: 2
Thread-1: 9
...
```

Ampliação da atividade

Nível 1 – Runnable

- Reimplementar usando Runnable.

Nível 2 – Nomeação de threads

```
t1.setName("Thread A");
```

Nível 3 – Controle de tempo

- Ajustar tempos de sleep.

Nível 4 – Múltiplas threads

- Criar 3 ou mais threads.

Nível 5 – Desafio extra (introdução à sincronização)

- Criar contador compartilhado.
- Observar inconsistências (condição de corrida).

5. Encerramento e orientações finais (20 minutos)

Discussão:

- Vantagens e desafios das threads
- Problemas de concorrência

Pergunta reflexiva:

“Quando vale a pena usar threads em um sistema?”

Reforço:

- start vs run
- execução paralela
- controle de fluxo

6. Orientações para o Relatório Final

O aluno deverá entregar um relatório (1 a 3 páginas):

Introdução

- Conceito de threads
- Execução concorrente

Desenvolvimento

- Explicação do programa
- Tipo de thread utilizado

Código comentado

```
// Inicia thread
t1.start();
```

Resultados

- Saída do programa

Conclusão

- Aprendizados

7. Critérios de avaliação

Critério	Peso	Descrição
Fundamentação teórica	2,0	Conceitos de threads
Implementação prática	3,0	Criação correta
Execução concorrente	2,0	Funcionamento
Organização do código	2,0	Clareza
Criatividade	1,0	Melhorias

Nota total: soma dos critérios avaliados.

8. Conclusão

Ao final da aula, os alunos estarão aptos a compreender e implementar **threads em Java**, desenvolvendo aplicações com execução concorrente. Esse conhecimento é essencial para sistemas modernos, que exigem desempenho, paralelismo e melhor aproveitamento de recursos computacionais.